

Modern JavaScript Concepts

Reference Guide & Practice Questions

This document covers essential ES6+ JavaScript concepts and functional array methods. Understanding these topics is highly important (V. IMP) for modern web development, particularly when working with frameworks like React, Node.js, and others.

Part 1: Functional Array Methods

1. `Array.prototype.map()`

The `map()` method creates a **new array** populated with the results of calling a provided function on every element in the calling array. It is highly useful for transforming data.

```
let array = [1, 2, 3, 4];
// Returns a new array, replacing odd numbers with null
let temp = array.map(num => num % 2 === 0 ? num : null);
// Output: [null, 2, null, 4]
```

2. `Array.prototype.filter()`

The `filter()` method creates a **shallow copy** of a portion of a given array, filtered down to just the elements from the given array that pass the test implemented by the provided function.

```
let filt = array.filter(num => num > 2);
// Output: [3, 4]
```

3. `Array.prototype.reduce()`

The `reduce()` method executes a user-supplied "reducer" callback function on each element of the array, passing in the return value from the calculation on the preceding element. It boils an array down to a **single value**.

```
// Summing all elements (0 is the initial value)
let add = array.reduce((sum, num) => sum + num, 0);

// Multiplying all elements (1 is the initial value)
let prod = array.reduce((product, num) => product * num, 1);
```

4. Array.prototype.find()

The `find()` method returns the **first element** in the provided array that satisfies the provided testing function. If no values satisfy the testing function, undefined is returned.

```
let search = array.find(num => num > 1);  
// Output: 2
```

5. Array.prototype.some()

The `some()` method tests whether **at least one element** in the array passes the test implemented by the provided function. It returns a boolean value (true or false).

```
let hasGreaterThanOne = array.some(num => num > 1);  
// Output: true
```

6. Array.prototype.every()

The `every()` method tests whether **all elements** in the array pass the test implemented by the provided function. It returns a boolean value.

```
let allGreaterThanOne = array.every(num => num >= 1);  
// Output: true
```

Part 2: Modern Syntax Features

7. Object Destructuring

Destructuring assignment makes it possible to unpack values from arrays, or properties from objects, into distinct variables. This leads to cleaner, more readable code.

```
const person = { name: "Jake", age: 22 };  
// Extracting variables directly from the object  
const { name, age } = person;  
console.log(name); // "Jake"
```

8. Spread Operator (...)

The **spread** operator allows an iterable (like an array or string) to be expanded in places where zero or more arguments or elements are expected. It is perfect for copying or merging arrays.

```
let array = [1, 2, 3, 4];
let array2 = [...array, 7, 8, 9];
// Output: [1, 2, 3, 4, 7, 8, 9]
```

9. Rest Parameters (...)

The **rest** parameter syntax allows a function to accept an indefinite number of arguments as an array, providing a way to represent variadic functions in JavaScript.

```
function sum(...numbers) {
  return numbers.reduce((total, num) => total + num, 0);
}
sum(10, 20, 40, 60); // Output: 130
```

10. Template Literals

Template literals are string literals enclosed by the backtick (`) character instead of double or single quotes. They allow for multi-line strings and string interpolation (embedding expressions directly).

```
const name = "Alice";
const age = 22;
console.log(`My name is ${name} and I am ${age} years old.`);
```

Part 3: Practice Questions

Test your knowledge with these 20 practice questions based on the topics above. Try to answer them before writing out the code.

Q1. What is the fundamental difference between the output of `filter()` and `find()`?

Answer space / Notes

Q2. If an array has 5 elements, and you call `map()` on it, how many elements will the resulting array have?

Answer space / Notes

Q3. Predict the output:
`[5, 10, 15].reduce((acc, curr) => acc + curr, 10);`

Answer space / Notes

Q4. True or False: `some()` will check every element in an array even if the first element evaluates to `true`.

Answer space / Notes

Q5. What will `[2, 4, 6].every(num => num % 2 === 0)` return?

Answer space / Notes

Q6. Write a line of code using object destructuring to extract `city` from `const user = { name: 'Bob', city: 'Paris' }`.

Answer space / Notes

Q7. Predict the output of array2:
`const arr1 = [1, 2]; const arr2 = [0, ...arr1, 3];`

Answer space / Notes

Q8. In the function signature `function greet(msg, ...names)`, what data type is `names` inside the function?

Answer space / Notes

Q9. How do you inject a JavaScript variable named `totalCost` into a template literal string?

Answer space / Notes

Q10. Does the `map()` method mutate (change) the original array it is called upon?

Answer space / Notes

Q11. Predict the output:

```
[1, 2, 3].map(n => n * 2).filter(n => n > 4);
```

Answer space / Notes

Q12. Which array method is best suited if you want to calculate the total price of all items in a shopping cart array?

Answer space / Notes

Q13. Predict the output:

```
['apple', 'banana', 'cherry'].find(fruit => fruit.startsWith('b'));
```

Answer space / Notes

Q14. What happens if `find()` cannot locate any element matching the condition?

Answer space / Notes

Q15. Can you use the rest parameter (`...`) at the beginning of a function's argument list? e.g., `function(...args, lastArg)`.

Answer space / Notes

Q16. If you have `const obj1 = { a: 1 }` and `const obj2 = { b: 2 }`, how can you use the spread operator to combine them into a new object `obj3`?

Answer space / Notes

Q17. Predict the output:
`[0, 10, 20].some(x => x > 15);`

Answer space / Notes

Q18. What is the output of `typeof `Hello World`` ?

Answer space / Notes

Q19. Predict the output of the following destructuring code:
`const { x } = { y: 10, z: 20 }; console.log(x);`

Answer space / Notes

Q20. You need to create an array containing only the names of users from an array of user objects. Which functional array method should you use?

Answer space / Notes